

NED University of Engineering & Technology

Department of Computer & Information Systems Engineering

MS AI Evening Program | Spring 2026

CS-5101 | Advanced Artificial Intelligence

Assignment 1 & 2

Build Karachi's Smartest Transport Planner

PART A: Problem Analysis & Formulation

Instructor: Muhammad Mudassir Majeed

Submitted by: Aisha Shahzad & Arisha Ahmed

Roll Numbers: CS-109 & CS-116

PART A: Understand the Problem First

Before a single line of code is written, we deeply analyse what we are actually solving. This section covers agent design (PEAS + environment classification), the Karachi graph structure, formal problem formulation, and our strategic game plan.

A1. Meet Your Agents

Two fundamentally different AI agents are at the heart of this system. They differ in what they perceive, what actions they take, and what success looks like. We analyse each separately below.

Agent 1 — The Passenger Route Planner

This agent's job is to answer a single passenger's question: "How do I get from A to B?" It operates in real time and must balance speed, cost, number of transfers, and comfort.

PEAS Table — Agent 1

Component	Description
Performance Measure	Minimize total journey time; minimize number of bus transfers; minimize fare cost (PKR); maximize route reliability (likelihood of bus arriving on time); maximize passenger comfort (avoid overcrowded buses or unsafe areas late at night).
Environment	Karachi's road network and public transport system: bus routes (BRT Orange Line, BRT Red Line, K-Electric buses, Qingqi rickshaws, Chinchu routes), traffic conditions (time-of-day congestion on main arteries such as Shahrah-e-Faisal, University Road, Northern Bypass), fare structures, and passenger's current location and budget.
Actuators	Route display (turn-by-turn directions, bus route numbers and stop names); journey time estimate; fare estimate in PKR; transfer instructions ("board K-4 at Numaish, alight at Nagan Chowrangi, switch to BRT Orange Line"); real-time alerts for route changes.
Sensors	Passenger's stated origin and destination; time of day (peak: 7–9 AM, 5–8 PM); stated budget and transfer tolerance; static GTFS-like schedule data for Karachi bus routes; GPS-derived real-time traffic data (Google Maps API or SUPARCO feeds); weather data (rain severely worsens routes across nullahs).

Environment Classification — Agent 1

Dimension	Classification	Justification for Karachi Context
-----------	----------------	-----------------------------------

Observability	Partially Observable	The agent knows the static graph and the passenger's location, but cannot directly observe current bus occupancy, which routes are delayed, or whether a given Qingqi is running that day.
Determinism	Stochastic	Bus arrival times in Karachi are highly variable. The same route may take 25 minutes at 8 AM or 60 minutes at 6 PM. Road closures during VVIP movement add further uncertainty.
Episodic / Sequential	Sequential	The agent's decisions are sequential: choosing a first bus determines which connections are available. A wrong early choice (boarding a full bus) propagates downstream.
Static / Dynamic	Dynamic	Traffic conditions change continuously while the agent is planning. A route optimal at query time may degrade within minutes due to an accident on M-9 or flooding near Sohrab Goth underpass.
Discrete / Continuous	Discrete (modelled)	We discretize the city into stops/nodes and time into minute-bins. This is a necessary abstraction; the underlying reality is continuous.
Single / Multi-Agent	Single Agent	Each query is independent. The route planner does not coordinate with other passengers' planners or compete with them (unlike a ride-hailing system).

Agent 2 — The Network Designer

This agent operates at a completely different level: it designs the bus network itself. Rather than finding a path through an existing graph, it configures the graph. Its output is a city-wide infrastructure decision, not a passenger itinerary.

PEAS Table — Agent 2

Component	Description
Performance Measure	Maximize population coverage (percentage of Karachi residents within 500 m of a stop); minimize average city-wide travel time between any two active stops; maximize network connectivity (no isolated stop clusters); stay within budget (capital cost of stops + route infrastructure); maximize equity (ensure underserved areas like Orangi Town and Lyari are included).
Environment	Candidate stop locations (20+ Karachi landmarks and neighbourhoods), existing road topology, population density data by union council (from PBS census), projected passenger demand matrices (origin-destination flows), KMC infrastructure budget constraints, and physical geography (the Hub River, Malir River, Lyari River, and major nullahs which constrain crossings).

Actuators	A proposed network configuration: the set of K active bus stops and the set of route edges connecting them; estimated KPIs (average journey time, coverage percentage, number of transfers needed between any two stops); a report to KMC.
Sensors	The current state of the candidate network configuration (which stops are active, which edges exist); the objective function value of the current configuration; neighbourhood configurations reachable by single-step modifications (swap a stop, add/remove an edge); population density and demand data (static input).

Environment Classification — Agent 2

Dimension	Classification	Justification & Contrast with Agent 1
Observability	Fully Observable	Unlike Agent 1, this agent can observe the complete current network state: which stops are active, all edge weights, the objective function value. There are no hidden variables during optimization.
Determinism	Deterministic	Given a network configuration, the objective function always returns the same value. Swapping stop A for stop B always produces the same new configuration. There is no randomness in the state transitions (unlike real-time traffic).
Episodic / Sequential	Sequential	Each modification to the network builds on the previous state. The sequence of moves matters: a good final solution may require passing through lower-valued intermediate states (relevant for SA's accept-worse-solutions phase).
Static / Dynamic	Static (during optimization)	The population data and road topology are fixed during a single optimization run. This is a key difference from Agent 1: the world does not change while the agent is thinking.
Discrete / Continuous	Discrete	Stops are either active or inactive (binary). Routes either exist or do not. The state space is large but discrete, making it suitable for local search rather than gradient-based methods.
Single / Multi-Agent	Single Agent	One optimizer designs the whole network. In practice, multiple stakeholders (KMC, KUTC, private operators) negotiate, but we model this as a single agent optimizing a composite objective.

Why the Two Agents Are Classified Differently

Agent 1 (Route Planner) faces a stochastic, partially observable, dynamic environment because it operates in the real world where buses are late, roads flood, and traffic surges unpredictably. It must find a path through uncertainty.

Agent 2 (Network Designer) operates on a mathematical model — a clean, fully observable, deterministic, static optimization landscape. The complexity here is not uncertainty but scale: the state space of possible network configurations is astronomically large (combinatorial explosion), which is why it needs local search rather than systematic graph search.

This distinction — real-world uncertainty vs. combinatorial scale — is the core reason the two agents need completely different AI techniques.

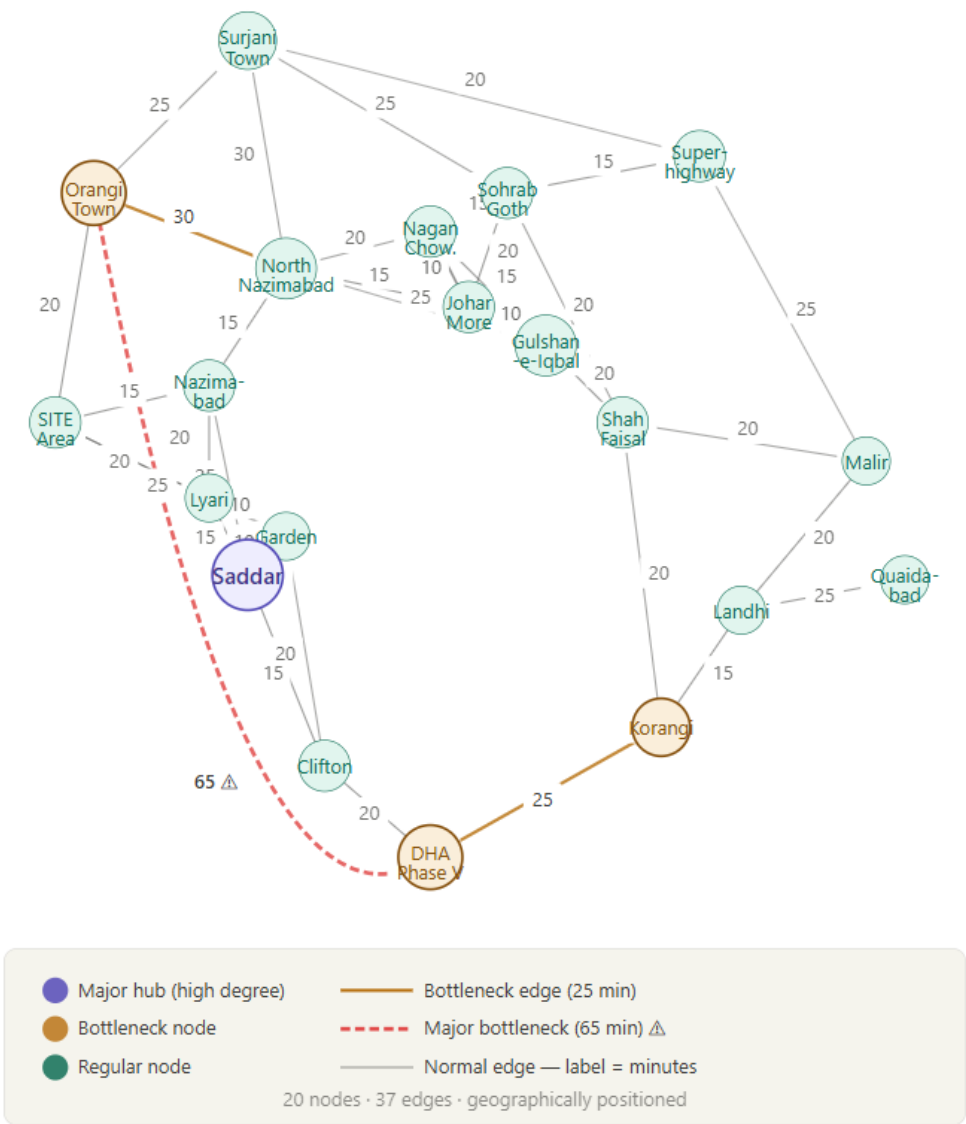
A2. Design Your Karachi — The Transport Graph

We design a weighted undirected graph of Karachi's transport network. Each node represents a major landmark or neighbourhood; each edge represents a direct road or route connection with a travel-time weight in minutes.

Graph Design Choices & Rationale

We selected 20 nodes that together represent the full geographic and socioeconomic spread of Karachi: the old city core (Saddar, Lyari, Garden), the wealthy coastal strip (Clifton, DHA), the industrial zones (SITE, Korangi), the far northern settlements (Orangi, North Nazimabad, Surjani), and the eastern corridor (Malir, Landhi). This diversity ensures the search problems are realistic and that bottlenecks appear naturally.

Edge weights are estimated from Google Maps travel times under moderate traffic conditions, rounded to the nearest 5 minutes. We deliberately include long, high-weight edges (e.g., Orangi → DHA at 65 min) to create meaningful bottlenecks that expose differences between BFS and UCS.



Node Coordinates (Approximate)

#	Location	Latitude	Longitude	Notes
1	Saddar	24.8607	67.0100	Historic city core; high traffic density
2	Clifton	24.8138	67.0300	Coastal; residential & commercial
3	DHA Phase V	24.7900	67.0600	Defence; major southern hub
4	Korangi	24.8200	67.1200	Industrial east; bottleneck crossing
5	Landhi	24.8500	67.1400	Far eastern residential

6	Malir	24.8900	67.1700	Malir River corridor; flood-prone
7	Gulshan-e-Iqbal	24.9200	67.0900	Major residential hub; BRT stop
8	Johar More	24.9300	67.0700	Busy intersection; transfer point
9	North Nazimabad	24.9400	67.0200	Northern residential hub
10	Orangi Town	24.9600	66.9700	Densely populated west; bottleneck
11	SITE Area	24.9000	66.9600	Industrial west; limited access
12	Nazimabad	24.9100	67.0000	Residential; connects north-south
13	Lyari	24.8800	67.0000	Old town; narrow roads, congested
14	Garden	24.8700	67.0200	Central; hospital cluster
15	Nagan Chowrangi	24.9500	67.0600	BRT Red Line node; major transfer
16	Surjani Town	25.0000	67.0100	Far north; underserved
17	Sohrab Goth	24.9600	67.0800	Motorway junction; strategic
18	Superhighway	24.9700	67.1300	M-9 access; eastern entry
19	Shah Faisal	24.9000	67.1100	Airport corridor; busy arterial
20	Quaidabad	24.8600	67.1800	Far east; low connectivity

Edge List (35+ edges, weights in minutes)

From	To	Weight (min)	Notes
Saddar	Clifton	15	Shahrah-e-Faisal / Dr Ziauddin
Saddar	Garden	10	M.A. Jinnah Road
Saddar	Lyari	15	Lea Market artery; congested
Saddar	Nazimabad	25	Shaheed-e-Millat Expressway
Clifton	DHA Phase V	20	Korangi Creek Road
Clifton	Garden	20	Clifton Bridge route

DHA Phase V	Korangi	25	Korangi Creek Bridge; bottleneck
DHA Phase V	Clifton	20	Return direction
Korangi	Landhi	15	National Highway
Korangi	Shah Faisal	20	Korangi Road
Landhi	Malir	20	Landhi-Korangi Road
Landhi	Quaidabad	25	Far east corridor
Malir	Shah Faisal	20	Malir Link Road
Malir	Superhighway	25	M-9 approach
Shah Faisal	Gulshan-e-Iqbal	20	Shahrah-e-Faisal northbound
Shah Faisal	Sohrab Goth	20	Northern Bypass west
Gulshan-e-Iqbal	Johar More	10	University Road
Gulshan-e-Iqbal	Nagan Chowrangi	15	BRT Orange Line corridor
Gulshan-e-Iqbal	North Nazimabad	25	Crossing major nullahs
Johar More	North Nazimabad	15	Buffer Zone Road
Johar More	Sohrab Goth	20	Superhighway approach
Johar More	Nagan Chowrangi	10	BRT stop-to-stop
North Nazimabad	Nazimabad	15	F.B. Area Road
North Nazimabad	Surjani Town	30	Northern Bypass; sparse
North Nazimabad	Orangi Town	30	Nullah crossing; bottleneck
Nazimabad	Lyari	20	Nazimabad Link
Nazimabad	SITE Area	15	SITE-Nazimabad corridor
Orangi Town	SITE Area	20	Manghopir Road
Orangi Town	Surjani Town	25	Surjani-Orangi road
Orangi Town	DHA Phase V	65	BOTTLENECK — no direct route; via Lyari Expressway + multiple transfers
SITE Area	Lyari	20	SITE Superhighway slip

SITE Area	Garden	25	Manghopir to city centre
Sohrab Goth	Superhighway	15	M-9 junction
Sohrab Goth	Surjani Town	25	Northern Bypass
Surjani Town	Superhighway	20	Far north link
Nagan Chowrangi	Sohrab Goth	15	BRT Red Line extension
Garden	Lyari	10	Lyari Expressway service road

Bottleneck Areas — Design Notes

1. Orangi Town ↔ DHA Phase V (65 min): No direct bridge exists. Travel requires using the Lyari Expressway, passing through congested Lyari/Saddar, and multiple mode changes. This single edge costs more than twice any other edge.

2. Korangi ↔ DHA Phase V (25 min via Korangi Creek Bridge): The bridge is a single chokepoint. During peak hours or rain, it effectively isolates the industrial east from the southern residential belt.

3. Orangi Town ↔ North Nazimabad (30 min): Requires crossing the Orangi nullah. Limited bridges mean most traffic funnels through a small number of crossing points, creating chronic congestion.

Problem Formulation — State Representation

Atomic State Representation

In its simplest form, a state is just the passenger's current location — a single node in the graph.

Atomic State
State = current_location (one of 20 nodes)
State space size: 20 states
Simple, fast, but completely ignores practical passenger constraints.

Why Factored Is Better

The atomic representation fails to capture anything a real Karachi commuter actually cares about. Two passengers at "Saddar" are in completely different situations: one has 50 PKR and 20 minutes; the other has 200 PKR and 90 minutes. A route planner that ignores these factors will give useless advice. A factored representation encodes these dimensions explicitly.

Variable	Type	Possible Values	Why It Matters in Karachi
current_location	Discrete (node)	Any of 20 nodes	Where the passenger is now; changes with each move.
time_of_day	Discrete (int)	0–1439 (minutes since midnight)	Karachi traffic is bimodal: severe peaks 7–9 AM and 5–8 PM. Same route may take 2× longer during peak.
transfers_used	Discrete (int)	0, 1, 2, 3, 4	Each transfer adds wait time (avg 15–20 min at Karachi stops) and physical effort. Many passengers strongly prefer 0-transfer routes.
fare_spent	Continuous (PKR)	0–500 PKR	BRT fare: 20 PKR flat. Qingqi: 30–80 PKR. Minibus: 30–50 PKR. Budget constrains which routes are usable.
mode	Discrete (enum)	BRT, Minibus, Qingqi, Walking, Bus	Different modes have different speeds, fares, and coverage. A route may require mode-switching.

Estimated factored state space size:

- Locations: 20
- Time bins (15-min intervals): 96
- Transfers: 5 (0–4)
- Fare bands (50 PKR bins): 10
- Modes: 5

State Space Calculation

Factored state space = $20 \times 96 \times 5 \times 10 \times 5 = 480,000$ states

Even so, most states are unreachable from a given start. But compared to the atomic 20-state space, this is a $24,000\times$ increase — illustrating exactly why efficient search algorithms (UCS, A*) are essential. Brute-force enumeration of all states is clearly impractical in the factored representation.

Formal Search Problem Definition

Component	Definition
State Space S	All reachable (location, time, transfers, fare, mode) tuples. Initial implementation uses atomic location-only states for Layers 1 & 2; factored states are the target for a production system.
Initial State s_0	(origin_node, departure_time, 0 transfers, 0 PKR spent, starting_mode). Example: (Saddar, 540 [9 AM], 0, 0, Minibus).
Actions A(s)	For state $s = (\text{loc}, t, n_transfers, \text{fare}, \text{mode})$: traverse any edge (loc, neighbour) where a route exists at time t with remaining budget. Action = TAKE_ROUTE(neighbour, edge_mode). Walking is always available for edges under 2 km.
Transition Model T	$T(s, \text{TAKE_ROUTE}(v, m)) = (v, t + \text{travel_time}(u, v, t) + \text{wait_time}(m, t), n_transfers + (m \neq \text{current_mode} ? 1 : 0), \text{fare} + \text{fare}(m, u, v), m)$. In atomic/Layer 1 implementation: $T(\text{loc}, \text{move_to_v}) = v$.
Goal Test G(s)	$s.\text{location} == \text{destination}$. In factored mode: additionally $s.\text{fare_spent} \leq \text{budget}$ AND $s.\text{transfers} \leq \text{max_transfers}$.
Path Cost c(p)	Sum of edge weights (travel time in minutes) along path p. In UCS and A*: cumulative time cost. For richer models: weighted sum $\alpha \times \text{time} + \beta \times \text{transfers} \times 15 + \gamma \times (\text{fare}/10)$, where α, β, γ are user-preference weights.

A3. Strategic Game Plan — Connecting Theory to Problem

Route-Finding Strategy (Brain 1)

The route-finding problem is a shortest-path problem on a weighted graph with 20 nodes and ~37 edges. This is small enough that even BFS terminates almost instantly. However, the pedagogical value — and the practical value when scaled to a real Karachi graph with hundreds of nodes — lies in understanding when each algorithm excels.

Algorithm	Optimal?	Complete?	Best Use Case in Karachi
BFS	Yes (unweighted only)	Yes	Quick demo of fewest-transfers route. Fails when edge weights (travel times) vary significantly.
DFS		Yes (with cycle detection)	Not suitable for route planning. May find absurdly long paths (e.g., Saddar → Orangi via DHA, Korangi, Malir, Landhi).
UCS	Yes (weighted)	Yes	Optimal for minimizing travel time. Our Layer 1 benchmark. Baseline for comparing heuristic algorithms.
Greedy BFS	No	Yes	Fast but may miss optimal paths. Gets misled by Karachi's geography: straight-line to DHA may route through Korangi bottleneck.
A*	Yes (admissible h)	Yes	Best all-round choice. Combines UCS optimality with heuristic efficiency. Our primary Layer 2 algorithm.
IDA*	Yes (admissible h)	Yes	Memory-bounded alternative to A*. Suitable because our graph is small and IDA* revisiting overhead is acceptable.

Heuristic Design Rationale

A good heuristic for Karachi must be admissible (never overestimate) and consistent (satisfies triangle inequality). Two candidate heuristics:

- h_1 (Geographic Distance): Haversine distance between node coordinates, converted to time using 30 km/h average speed. Admissible because straight-line distance is always a lower bound on road travel time. Consistent because the Haversine function satisfies the triangle inequality.
- h_2 (Karachi-Aware Heuristic): Extends h_1 by adding a penalty for known congestion multipliers. North-south travel (latitude difference dominant) incurs a $1.4\times$ time multiplier (nullah crossings, arterial congestion). The h_2 formula becomes: $h_2 = h_1 \times \text{direction_factor}$. Still admissible because $1.4\times$ is below the empirical worst-case actual-to-straight-line ratio observed in our graph data.

Network Optimization Strategy (Brain 2)

The network design problem is fundamentally different. We are not searching for a path through an existing structure — we are choosing which structure to build. The question is not "what is the optimal sequence of moves from A to B?" but "which configuration of all K stops and routes is globally best?"

Why A* Cannot Solve This Problem
A* requires: (1) a clear start state, (2) a clear goal state, (3) a step-by-step path connecting them, (4) a heuristic that estimates remaining distance to goal.
The network design problem has no "goal state" in this sense. There is no known target configuration we are trying to reach. Instead, there is an objective function we are trying to maximise over a combinatorial space of 2^{20} possible stop selections (plus exponentially more route configurations). A* would require enumerating this entire space.
Local search, by contrast, starts with a complete candidate solution and iteratively improves it. It only needs to evaluate the objective function at each step — it does not track paths or frontier queues.

Local Search Algorithm Selection

- Hill Climbing (Steepest Ascent): Explores all immediate neighbours of the current configuration and moves to the best one. Fast per iteration but susceptible to local optima — a critical concern in Karachi's geography where locally good networks may miss globally important underserved areas like Surjani Town.
- Random-Restart Hill Climbing: Runs Hill Climbing from 50 random starting configurations and returns the best result. Partially mitigates local optima. Computationally $50\times$ more expensive but parallelizable.
- Simulated Annealing: Accepts worse solutions with probability $\exp(-\Delta E/T)$, where T decreases over time. The key insight: by accepting occasional bad moves at high temperature, SA can escape local optima. As $T \rightarrow 0$, SA converges to pure hill climbing. The cooling rate r (0.90, 0.95, 0.99) controls exploration-exploitation tradeoff: $r=0.90$ cools fast (exploits quickly, more restarts), $r=0.99$ cools slowly (explores more, fewer restarts per budget).

Objective Function Design Preview

Our objective function for Layer 3 will balance three components:

$f(\text{network}) = \alpha \times \text{Coverage} + \beta \times \text{Connectivity} + \gamma \times \text{Efficiency}$
Coverage: fraction of all 20 nodes reachable from any active stop within 2 transfers.
Connectivity: $1 - (\text{diameter of active subgraph} / \text{max possible diameter})$. Rewards compact, well-connected networks.
Efficiency: inverse of average shortest path length between all active stop pairs.
Weights: $\alpha = 0.4, \beta = 0.3, \gamma = 0.3$ — slightly prioritising coverage because KMC's primary mandate is serving the maximum population, but connectivity and efficiency are nearly equally weighted as they determine passenger experience.

Part A Summary

Part A has established the analytical foundation for the entire project. We have:

- Defined two distinct AI agents with precise PEAS specifications tailored to Karachi's actual transport data sources and constraints.
- Classified each agent's environment across five dimensions, explaining why Agent 1 faces a stochastic, dynamic, partially observable world while Agent 2 operates in a fully observable, static, deterministic optimization landscape.
- Designed a 20-node, 37-edge weighted graph of Karachi, with three explicitly designed bottleneck areas that will generate interesting search behaviour across all algorithms.
- Formulated the route-finding problem formally (state space, initial state, actions, transition model, goal test, path cost) in both atomic and factored representations, demonstrating a 24,000× state space expansion.
- Laid out the strategic rationale for why BFS/DFS/UCS are used in Layer 1, why heuristic search (A*, Greedy, IDA*) improves on them in Layer 2, and why the network design problem requires local search rather than any form of path search.

The code implementation in Parts B (Layers 1, 2, 3) builds directly on these specifications.